

Floently Learn Material Generation, Verification, and Certification Standard

Document type: Normative internal standard

Short name: MGVCS

Full name: Floently Learn Material Generation, Verification, and Certification Standard

1. Purpose

This document defines the canonical process by which Floently Learn materials are:

1. **generated,**
2. **recovered from legacy or distorted sources,**
3. **verified and sanitized,**
4. **certified for release,** and
5. **injected into the canonical bank.**

It exists to remove ambiguity from material authoring, transformation, validation, and publication.

This standard is intended to be detailed enough that a new engineer, reviewer, or pipeline operator can read it and reproduce the process without guessing, improvising, or creating a parallel workflow.

2. Scope

This standard applies to all Floently Learn materials that are intended for the canonical bank, including but not limited to:

- vocabulary cards,
- phrase/sentence cards,
- grammar cards,
- profession-specific language materials,
- YKI study materials,
- release-candidate materials,
- recovered legacy materials,
- AI-assisted recovered materials.

This standard does **not** allow raw generated material, donor material, or legacy material to be used directly by runtime.

Runtime consumes Accepted Published Structure (APS) only.

3. Core principle

Floently Learn uses a **three-layer material model**:

1. **MGI** — Minimum Generated Input
2. **AID** — Augmenting Input Data
3. **APS** — Accepted Published Structure

3.1 MGI

MGI is the smallest acceptable authoring payload.

It contains only the minimum item content needed to represent a learning item.

Examples:

- Vocabulary: `term` + `meaning`
- Phrase: `sentence` + `meaning`
- Grammar: `pattern` + `usage` + `example_fi` + `meaning`

3.2 AID

AID is the contextual and governing data that helps the pipeline interpret MGI safely.

AID includes:

- batch manifest,
- source lineage,
- inferred or explicit profession,
- domain,
- level band,
- path,
- confidence metadata,
- recovery metadata,
- optional trusted hints.

3.3 APS

APS is the rich runtime/publication format used by the canonical bank.

APS includes:

- stable IDs,
- publication metadata,
- quality metadata,
- provenance,
- content envelope,
- prompts,
- follow-ups,
- validation settings,
- tags,

- release/certification state.

Only APS may enter the canonical bank for runtime use.

4. Normative rules

The following rules are mandatory.

Rule 1 — Manifest-first generation is the preferred authoring format

Human or AI generators should first produce a **batch manifest**, followed by the list of generated items that conform to that manifest.

Rule 2 — Raw sources may enter the pipeline, but not runtime

Any source may enter the sanitation pipeline, including distorted, legacy, or donor material.

No raw source may bypass the pipeline and enter the canonical bank directly.

Rule 3 — APS is generated by the pipeline

APS is not the preferred authoring format for large-scale generation.

APS is produced by the pipeline from MGI + AID after validation.

Rule 4 — Deterministic logic remains the pipeline authority

Deterministic validation, schema enforcement, packaging, and release rules remain authoritative.

Rule 5 — AI may assist, but may not certify by itself

AI may:

- classify,
- recover,
- verify,
- compare,
- detect likely errors,
- support pre-human review.

AI may not, by itself:

- declare publication validity,
- mark materials as reviewed,
- mark YKI material as certified,
- promote material to canonical bank,
- override hard validation failures.

Rule 6 — Human review remains required where confidence is insufficient or risk is high

Human review is required for:

- low-confidence recovered items,
 - high-impact release subsets,
 - YKI-sensitive or certification-sensitive material,
 - domain-critical professional material,
 - disputed duplicate clusters,
 - AI-flagged edge cases.
-

5. Canonical source and destination model

5.1 Inputs

Inputs may include:

- clean manifest-first MGI batches,
- canonical live contributors,
- legacy donor banks,
- distorted or partially structured material,
- compiled donor material,
- YKI canonical material trees,
- manually authored rescue material,
- AI-generated MGI batches.

5.2 Release-candidate destination

The pipeline writes to **temporary release-candidate directories** first.

Example structure:

```
apps/backend/materials/cards/  
  published/  
    release_candidates/  
      <run_id>/  
        manifest.json  
        foundation_yki/  
        doctor/  
        nurse/  
        practical_nurse/  
        reports/  
        rejected/  
        metrics.json  
      releases/  
        <approved_release_id>/
```

YKI candidate structure may follow the same controlled pattern.

5.3 Canonical bank injection

No release candidate is injected into the canonical bank until:

- schema validation passes,
 - duplicate thresholds pass,
 - sanitation rules pass,
 - release metrics pass,
 - human review passes where required,
 - certification status is assigned correctly.
-

6. Preferred authoring format

The preferred authoring format is:

1. one batch manifest,
 2. followed by one homogeneous item list.
-

7. Batch manifest specification (AID)

A batch manifest is the governing header for a generated batch.

7.1 Required fields

```
{
  "content_type": "vocabulary_card",
  "language": "fi",
  "path": "professional/doctor/words",
  "domain": "medical_work_finnish",
  "profession": "doctor",
  "level_band": "B1_B2",
  "source_id": "source.generated.professional.doctor.vocabulary"
}
```

7.2 Field meanings

- `content_type` : the intended item family, e.g. `vocabulary_card` , `phrase_card` , `grammar_card`
- `language` : authoring language of the main material, typically `fi`
- `path` : logical pathway or bank placement hint
- `domain` : content domain, e.g. `medical_work_finnish` , `patient_communication`
- `profession` : profession track or `none`
- `level_band` : pedagogical band, e.g. `A1_A2` , `B1_B2` , `C1_C2`
- `source_id` : lineage identifier for traceability

7.3 Optional fields

```
{
  "authoring_note": "generated doctor vocabulary set",
  "generator": "human",
  "generator_model": null,
  "country_context": "Finland",
  "audience": "adult_learners",
  "review_priority": "high",
  "batch_label": "doctor_vocab_batch_014"
}
```

8. Minimum Generated Input (MGI) specification

8.1 Vocabulary MGI

```
[
  { "term": "vieroitus", "meaning": "withdrawal" },
  { "term": "haavanhoito", "meaning": "wound care" }
]
```

8.2 Phrase MGI

```
[
  {
    "sentence": "Voisitko kuvailla kipua tarkemmin?",
    "meaning": "Could you describe the pain in more detail?"
  }
]
```

8.3 Grammar MGI

```
[
  {
    "pattern": "passive present",
    "usage": "used when the doer is unknown or unimportant",
    "example_fi": "Potilas siirretään osastolle.",
    "meaning": "The patient is moved to the ward."
  }
]
```

8.4 Why grammar requires more input than vocabulary and phrases

Vocabulary and phrase cards can safely be scaffolded heavily by the pipeline.

Grammar cards require slightly richer input because the pipeline must not invent:

- misleading grammar usage,
- weak grammar examples,
- contextually false teaching material.

Therefore the minimum grammar item includes:

- `pattern`,
- `usage`,
- `example_fi`,
- `meaning`.

9. Accepted Published Structure (APS)

APS is the canonical runtime/publication format.

Example:

```
{
  "id": "card.vocab.general.vieroitus.0089",
  "version": 1,
  "path": "general",
  "domain": "general_finnish",
  "profession": {
    "track": "none",
    "slug": null,
    "label": null
  },
  "level_band": "C1_C2",
  "difficulty": "stretch",
  "language": "fi",
  "source": {
    "source_id": "source.kielitaika.normalized.cards",
    "kind": "imported_workspace",
    "origin_path": "apps/backend/materials/cards/imports/kielitaika_normalized/cards_authority.json",
    "authoring_note": "source.generated.general.vocabulary"
  },
  "quality": {
    "status": "reviewed",
    "reviewer": "material-convergence",
    "validation_checks": [
      "normalized_precheck",
      "schema_validation",
      "import_mapping"
    ],
    "quality_score": 0.9
  },
}
```

```

"tags": ["noun", "withdrawal", "c1.id", "stretch"],
"publication": {
  "state": "validated",
  "version_tag": "cards_kielitaika_import_2026_04",
  "manifest_ref": "manifest.kielitaika.normalized.cards",
  "validation_passed": true,
  "published_at": null,
  "archived_at": null
},
"content_type": "vocabulary_card",
"content": {
  "front": {
    "term": "vieroitus",
    "lemma": "vieroitus",
    "part_of_speech": "noun"
  },
  "back": {
    "recall_prompt": "What does 'vieroitus' mean?",
    "gloss": "withdrawal",
    "example_sentence": null
  },
  "prompt_family": "vocabulary_memory",
  "follow_ups": [
    {
      "variant_type": "recognition_mcq",
      "prompt": "What does 'vieroitus' mean?",
      "options": [
        { "option_id": "o1", "text": "addiction", "explanation": null },
        { "option_id": "o2", "text": "withdrawal", "explanation": null },
        { "option_id": "o3", "text": "treatment", "explanation": null },
        { "option_id": "o4", "text": "detox", "explanation": null }
      ],
      "answer_key": "o2",
      "accepted_variants": [],
      "evaluation_mode": "option_id"
    }
  ],
  "explanation": {
    "summary": "Imported vocabulary card for 'vieroitus'.",
    "example": "vieroitus"
  },
  "audio": null,
  "validation": {
    "case_sensitive": false,
    "normalize_whitespace": true,
    "allow_partial_credit": false
  }
}
}

```

APS is the only structure admitted into the canonical bank.

10. What the pipeline generates automatically

The pipeline may generate or derive the following from MGI + AID:

10.1 Envelope and governance

- stable ID,
- version,
- profession block,
- path,
- domain,
- level band,
- difficulty,
- source metadata,
- initial quality metadata,
- initial publication metadata,
- tags,
- validation settings.

10.2 Content scaffolding

The pipeline may generate:

- recall prompts,
- prompt family,
- follow-up structure,
- recognition MCQs,
- typed recall shells,
- explanation summaries,
- normal validation settings,
- example fallbacks where explicitly allowed.

10.3 The pipeline must not silently invent critical truth

The pipeline must not silently invent:

- publication validity,
- review completion,
- YKI certification truth,
- unreliable profession alignment,
- unreliable grammar explanations,
- misleading examples,
- fabricated provenance.

11. ID policy

Floently Learn uses **deterministic IDs**, not random UUIDv4, for published materials whenever practical.

11.1 Preferred approach

Use stable, semantic IDs or UUIDv5-style deterministic IDs derived from normalized content.

11.2 Recommended semantic pattern

```
card.<type>.<pathway>.<profession>.<normalized_key>.<suffix>
```

Examples:

```
card.vocab.professional.doctor.vieroitus.01  
card.phrase.professional.nurse.kuvailla-kipua-tarkemmin.01  
card.grammar.professional.practical_nurse.passive-present.01
```

11.3 Why random IDs are not preferred

Random IDs make regeneration, deduplication, maintenance, and diffing harder.

Deterministic IDs make it easier to:

- compare runs,
- detect duplicates,
- regenerate APS predictably,
- preserve lineage,
- repair release candidates without unstable identity churn.

12. Pipeline stages

The canonical pipeline consists of the following stages.

Stage 0 — Intake

The pipeline inventories all sources.

Outputs:

- source inventory,
- source family classification,
- file-type classification,
- shard plan.

Stage 1 — AID detection and recovery

The pipeline detects or recovers AID.

Best case:

- manifest is explicit.

Recovery case:

- infer from path,
- infer from filename,
- infer from repeated fields,
- infer from directory structure,
- infer from source registry.

Outputs:

- recovered manifest,
- manifest confidence,
- manifest origin (`explicit` , `inferred` , `ai_supported`).

Stage 2 — MGI extraction

The pipeline extracts the minimum item payload.

It normalizes variants such as:

- `term` / `word` / `front` ,
- `meaning` / `gloss` / `translation` ,
- `sentence` / `phrase` / `fi` ,
- `pattern` / `topic` ,
- `example_fi` / `example` ,
- plain text pairs.

Outputs:

- canonical MGI candidate,
- extraction confidence,
- uncertainty flags.

Stage 3 — AI-assisted recovery (optional but supported)

AI may assist when deterministic extraction is uncertain.

AI tasks may include:

- content-type classification,
- structured extraction,
- AID hint suggestion,
- likely profession/domain fit,
- uncertainty flagging.

AI output is provisional and must re-enter deterministic validation.

Stage 4 — Normalization

The pipeline normalizes:

- keys,
- profession values,
- domains,
- levels,
- punctuation,
- whitespace,
- language markers,
- dedupe signatures,
- text canonicalization.

Stage 5 — Duplicate analysis

The pipeline performs:

- exact duplicate detection,
- normalized duplicate detection,
- near-duplicate candidate generation,
- cluster formation,
- cluster adjudication.

Stage 6 — APS generation

The pipeline transforms normalized MGI + AID into APS.

Stage 7 — Sanitation and scoring

The pipeline rejects or quarantines materials that fail release safety.

Stage 8 — Human review

Flagged items and sampled high-impact subsets are reviewed.

Stage 9 — Certification and release packaging

Only approved items are packaged into release candidates and promoted after sign-off.

13. How AI sits in the pipeline

AI is a **support layer**, not the source of truth.

13.1 Sweet spot placement

AI is best placed in four areas:

1. **Batch-level AID verification**
2. **Uncertain item extraction and recovery**

- 3. Near-duplicate adjudication
- 4. Pre-human release screening

13.2 AI must not be used as runtime authority

AI must not sit inside live runtime and must not be required for runtime to interpret material.

13.3 AI must not be the final certifier

AI may recommend:

- accept,
- reject,
- review,
- cluster merge,
- confidence score.

AI may not alone mark:

- certified,
 - validated for publication,
 - reviewed,
 - published.
-

14. Processing model for medium to extremely large datasets

The pipeline must not process the entire corpus as a single in-memory job.

14.1 Required operating model

Use **sharded, checkpointed, queue-based processing**.

14.2 Natural shard boundaries

Preferred shard boundaries:

- one manifest batch,
- one content type,
- one profession,
- one domain,
- one level band,
- one source family,
- one file.

14.3 Stage outputs must be persisted

Each stage should write persistent outputs such as:

- `inventory.json`
- `manifest_recovery.json`

- `normalized_records.jsonl`
- `duplicate_candidates.jsonl`
- `ai_reviews.jsonl`
- `aps_output.jsonl`
- `validation_report.json`
- `release_decisions.json`

14.4 Append-only preferred formats

Preferred intermediate formats:

- JSONL,
- SQLite,
- Parquet,
- database staging tables.

Avoid giant monolithic JSON for large runs where practical.

14.5 Pipeline lanes

Recommended processing lanes:

Lane A — Clean lane

For clean manifest-first MGI batches.

Lane B — Salvage lane

For messy or legacy material.

Lane C — Adjudication lane

For duplicate clusters, low-confidence items, and release-sensitive edge cases.

15. Legacy and distorted material recovery policy

The pipeline must attempt to salvage legacy material when reasonable.

15.1 Recovery objective

The recovery goal is to determine:

- MGI,
- AID,
- source lineage,
- confidence.

15.2 Recovery levels

Level A — Exact recovery

The structure is clear and can be normalized without ambiguity.

Level B — Probable recovery

The structure is recoverable with minor inference.

Level C — Uncertain recovery

AI assistance and/or human review is required.

15.3 Unsupported material

Material that cannot yield trustworthy MGI or minimally coherent AID must be rejected or quarantined.

16. Sanitation rules

The pipeline must reject or quarantine items that are:

- malformed,
 - corrupted,
 - duplicate,
 - near-duplicate low-value repetition,
 - generic filler,
 - wrong-language leakage,
 - profession/domain mismatch,
 - implausible grammar examples,
 - non-communicative descriptive stock,
 - low-value donor noise,
 - structurally incomplete beyond recovery.
-

17. Certification states

Certification must be explicit and truthful.

17.1 Recommended material states

```
raw_ingested
manifest_recovered
normalized
candidate_generated
validation_passed
needs_human_review
reviewed
```

```
release_candidate
certified_for_canonical_bank
published
archived
rejected
quarantined
```

17.2 Important distinction

`validated` is not the same as `certified`.

- **Validated** means the item passed technical and policy checks.
- **Certified** means the item has passed the release process required for canonical bank admission.

17.3 YKI-sensitive rule

No YKI material may be labeled in a way that implies certification or verified authority unless that status is actually true.

18. Human review requirements

Human review is required when any of the following apply:

- confidence is below release threshold,
- AI and deterministic logic disagree materially,
- duplicate adjudication remains uncertain,
- the item is domain-critical,
- the item is YKI-sensitive,
- the batch is strategically important,
- the release candidate is sampled for quality verification.

Human reviewers must be able to inspect:

- original raw fragment,
 - recovered MGI,
 - recovered AID,
 - AI notes,
 - validation results,
 - APS candidate,
 - duplicate cluster if relevant,
 - provenance path.
-

19. Certification before canonical bank injection

Before a release candidate may be injected into the canonical bank, the following must be true:

1. APS schema is valid.
2. Required provenance fields are present.

3. Duplicate thresholds are passed.
 4. Sanitation rules are passed.
 5. Release metrics are acceptable.
 6. Human review is complete where required.
 7. Certification status is assigned truthfully.
 8. Promotion is recorded in the release manifest.
-

20. Promotion procedure

20.1 Build candidate

Pipeline writes to release-candidate directories only.

20.2 Validate candidate

Run:

- schema checks,
- duplicate checks,
- domain coverage checks,
- malformed token checks,
- profession integrity checks,
- YKI integrity checks where applicable,
- runtime smoke-load checks.

20.3 Review candidate

Human review is completed on required subsets.

20.4 Promote candidate

After approval, the candidate is promoted into the canonical bank through a controlled manifest-governed release process.

20.5 Archive replaced bank if necessary

Prior release state should be archived to support rollback and audit.

21. Reproducibility and auditability

Every run must preserve enough data to reproduce or inspect the result.

A pipeline run should preserve:

- run ID,
- source inventory,
- manifest recovery output,
- normalized records,

- duplicate reports,
- AI review artifacts,
- APS candidates,
- validation reports,
- release decisions,
- final release manifest.

22. Minimal implementable examples

22.1 Preferred input example — vocabulary batch

```
{
  "manifest": {
    "content_type": "vocabulary_card",
    "language": "fi",
    "path": "professional/doctor/words",
    "domain": "medical_work_finnish",
    "profession": "doctor",
    "level_band": "B1_B2",
    "source_id": "source.generated.professional.doctor.vocabulary"
  },
  "items": [
    { "term": "vieroitus", "meaning": "withdrawal" },
    { "term": "haavanhoito", "meaning": "wound care" }
  ]
}
```

22.2 Distorted legacy input example

```
[
  { "word": "vieroitus", "gloss": "withdrawal" },
  { "word": "haavanhoito", "gloss": "wound care" }
]
```

Recovered internally as:

```
{
  "aid": {
    "content_type": "vocabulary_card",
    "language": "fi",
    "profession": "doctor",
    "domain": "medical_work_finnish",
    "level_band": "B1_B2",
    "manifest_origin": "inferred",
    "inference_confidence": 0.86
  },
  "mgi_items": [
```

```
{ "term": "vieroitus", "meaning": "withdrawal" },  
  { "term": "haavanhoito", "meaning": "wound care" }  
]  
}
```

23. Operational summary

The approved Floently Learn material contract is:

1. Authors or generators create **MGI** under a batch manifest.
2. The pipeline reads or recovers **AID**.
3. The pipeline extracts or recovers **MGI**.
4. Deterministic logic remains authoritative.
5. AI supports verification and recovery where appropriate.
6. The pipeline generates **APS**.
7. Validation, sanitation, and review occur.
8. Only certified release candidates are injected into the canonical bank.
9. Runtime consumes **APS only**.

24. Final statement

Floently Learn does **not** treat generated material as publication-ready by default.

Floently Learn treats material generation, recovery, verification, and certification as a governed process.

This standard exists to ensure that:

- generated scale does not damage quality,
- legacy recovery does not corrupt the bank,
- AI assistance improves precision without becoming the authority,
- canonical bank admission remains traceable, reproducible, and trustworthy.

25. Status line for internal use

Official internal statement:

All Floently Learn materials must pass through the MGI → AID → APS pipeline, then validation, review, certification, and controlled canonical-bank injection before runtime use.